Identification of Missing Gate Faults in Reversible Circuits by a Unity Method

1st Dimpimoni Kalita
Department of CSE, TSSOT
Assam University
Silchar-788011, Assam, India
monidimpi5@gmail.com

2nd Mousum Handique
Department of CSE, TSSOT
Assam University
Silchar-788011, Assam, India
mousum.smit@gmail.com

2024 7th International Conference on Circuit Power and Computing Technologies (ICCPCT) | 979-8-3503-7281-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCPCT61902.2024.10672995

***Abstract*—Reversible computing offers a remarkable capability of minimizing heat dissipation in computing systems. The primary advantage lies in its potential to mitigate heat dissipation in computing machinery. This paper describes an approach for recognizing missing gate faults (MGFs) in reversible circuits. The approach that has been put forward utilizes the unity method to effectively identify all types of MGFs—single, multiple, partial, and repeated gate faults. In this work, the unity method generates the single test pattern at the initial level of the circuit using a backtracking process that undergoes generation and conversion of the unity test pattern to an augmented test pattern, which is further reconstructed. A generated test pattern is crucial for pinpointing respective faults, resulting in a significant reduction in computational overhead. Through experimentation on several reversible benchmark circuits, we validate the efficacy of our approach and also provide a correlative evaluation of the existing work.**

***Index Terms*—Missing Gate Faults (MGFs), unity test pattern, augmented test pattern, test pattern, fault detection**

I. Introduction

The idea of computational reversibility is a strategy that is aimed at reducing or possibly eliminating the energy consumption associated with computational tasks [1]. Efficiently managing energy loss represents a crucial concern within digital logic design. Energy dissipation can stem from the non-ideal behavior of switches or devices. With technological advancements and the adoption of novel fabrication processes in recent decades, the intricacy of addressing heat loss and dissipation issues has intensified within deep-submicron integrated circuits (ICs) [2]. Reversible computing is characterized by its ability to process information without loss, theoretically resulting in zero power dissipation. This field has emerged as a thriving area of research in recent times. Reversible circuits executing reversible computations are regarded as a distinctive subset of quantum circuits. Quantum computing, known for its remarkable enhancement over classical computing, enables vast parallel computations within a single step. It offers significantly faster solutions to various problems, including prime factorization, compared to non-quantum methodologies [3]. Reversible circuits consist of n inputs and n outputs, ensuring that each input pattern corresponds uniquely to an output pattern, thereby allowing for the retrieval of inputs from outputs [4].

Frequent operation at high frequencies and the compact arrangement of circuit components increase the likelihood of errors occurring within the circuit. Consequently, testing becomes crucial to ensure the reliability and consistent performance of the circuit. Any undesired characteristic that emerges in the circuit, disrupting its functionality or behavior, is referred to as a fault or error. To effectively evaluate physical defects, abstract representations are indispensable. These conceptual models are constructed using a mathematical structure referred to as a fault model. A fault within a circuit refers to an error or undesirable characteristic that disrupts the intended utility and performance, whether over an indefinite duration or a limited time-frame. Consequently, testing becomes crucial for pinpointing such imperfections or errors within the circuit, ensuring its reliability and consistent operation [5].

This paper introduces a testing methodology for reversible circuits incorporating gates from the multiple-controlled-toffoli (MCT) library. Within this study, we focus on the MGF model, encompassing single missing gate fault (SMGF), multiple missing gate fault (MMGF), repeated gate fault (RGF), and partial missing gate fault (PMGF). Here, the method initiates for generating the unity test pattern by applying the logic value 1 for each input line of the reversible circuit. Next, the unity test pattern is converted to the augmented test pattern by applying the binary summation process. Now, the generated augmented test pattern is reconstructed by ignoring simply the least significant bit (LSB) to maintain the n count of input lines of the circuit. After reconstructing the augmented test pattern, the backtracking process is applied from the final output gate to the first gate of the circuit to generate the test pattern at the circuit's initial input level, which identifies the different types of faults under the MGF model.

The main contribution of this paper includes developing a single test pattern with the help of the unity method that recognizes all types of MGFs.

The remaining portion of the paper is organized as follows: Section II includes a brief background that relates to the reversible logic circuits, the MGF model, and a discussion of some existing works. Section III provides the details of the proposed methodology. Section IV and Section V include the experimental findings along with the comparative analysis with earlier work and conclusion remarks, respectively.

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.

979-8-3503-7281-6/24/$31.00 ©2024 IEEE

II. Background

A. Reversible Logic

A reversible function operates such that every individual input vector corresponds uniquely to an output vector, forming a permutation. It ensures that the original input vector can be accurately reconstructed given an output vector. In traditional non-reversible logic design, signal propagation in reverse can be hindered by the irreversible nature of commonly used gates. Nonetheless, any non-reversible Boolean function can be translated into a reversible circuit by incorporating additional lines for garbage and employing reversible gates. This transformation facilitates the backward propagation of signals [2]. A function $f{:}X \rightarrow Y$ is considered surjective if every element in the codomain Y is mapped to by at least one element in the domain X. In essence, for every y in Y, there exists an x in X such that $f(x)$=y. Conversely, a function $f{:}X \rightarrow Y$ is injective if distinct elements in the domain X map to distinct elements in the codomain Y.

B. Reversible Gates and Circuits

Reversible circuits employ specific gates known as reversible gates. These gates adhere to reversible logic, meaning they must be bijective, with a similar count of inputs and outputs, maintaining a one-to-one relationship between them. The Multiple Controlled Toffoli (MCT) or k-CNOT library consists of three fundamental gates: NOT, Feynman, and Toffoli gates [6]. The composition of MCT gates includes a control point ($\bullet$) and a target point ($\oplus$).

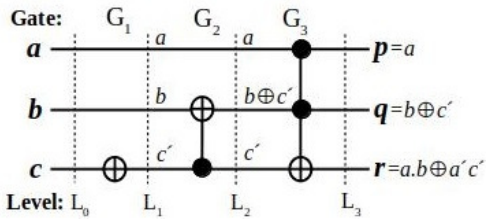

Fig. 1. Demonstration of k-CNOT circuit

A reversible circuit with n inputs and n outputs is a combination of reversible gates that adhere to the principle of linear cascading [7]. It entails the sequential processing of primary input, passing through each gate in succession. The outcome of each gate serves as the input for the next gate at each level, concluding the final one. Additionally, reversible circuits prohibit feedback and fan-out connections [8]. The pictorial representation of a reversible circuit appears in Fig. 1 in which the circuit consists of three lines consisting of primary inputs a, b, c and producing primary outputs p, q and r. Also the circuit contains three gates G_1, G_2, G_3 specifying four levels as L_0, L_1, L_2, and L_3. Each gate is fed in between two levels. Here, the first gate G_1 is the NOT gate, which consists of only a target point and is present in line c, thereby forming the output of the NOT gate is c'. The gate G_2 is the Feynman or 1-CNOT gate consisting of one control and one target point. The outputs of the gate G_2 are a, $b \oplus c'$ and c' for the input lines a, b and c, respectively at level L_2, as depicted in Fig. 1. The primary gate G_3 of the circuit is the Toffoli or 2-CNOT gate containing two control points and one target point, thereby forming the primary outputs p=a, q=$b \oplus c'$ and r=$a.b \oplus a'c'$, as shown in Fig. 1.

C. Missing Gate Fault (MGF) Model

A fault within a circuit denotes an anomaly that disrupts its normal functional behavior. The introduction of fault models helps in finding the flaws in reversible circuits and assesses the efficiency of testing. The fault model aligns with the testing objectives and assesses the specific types of faults present. The MGF model classifications encompass SMGF, MMGF, RGF, and PMGF [1], [4], [17]. SMGF is the missing of a single gate from the circuit. Let us consider in Fig. 1, if gate G_1 or G_2 or G_3 get missed then it will be called as SMGF. SMGF count is similar to the gate count of the reversible circuit. MMGF is the consecutive missing of two or more gates together. In Fig. 1, missing of gates $\{G_1, G_2\}$, $\{G_2, G_3\}$, $\{G_1, G_2, G_3\}$ will be termed as MMGFs. A reversible circuit containing N number of gates will have $[N.(N-1)/2]$ number of MMGFs. RGF is the repetition of any gate for any number of times, and here, we have considered the repetition of each gate one more time. Thus, the RGF count equals the SMGF count. PMGF is the missing control point(s) from a gate present in the circuit. Here, we have considered the missing control points at one time. The overall PMGF count depends on the count of control points in the circuit.

D. Related Work

The paper [7] 2008 presented two approaches for obtaining a comprehensive test set specifically targeting MGFs within reversible circuits. Initially, the division of circuits takes place to determine the complete test set, which is further minimized. The work in [2] demonstrates that introducing just one extra control line and duplicating each k-CNOT gate in an $(n \times n)$ reversible circuit makes it feasible to create a design that is easily testable. This design accommodates a universal test set (UTS) of size (n+1), capable of identifying all instances of SMGFs, RGFs, and PMGFs within the circuit. In 2014, the work in [9] focused on SMGF using the Boolean difference method. A Boolean generator is developed by the Boolean difference method to generate a test set for SMGF detection. In 2016, the authors in [10] introduced a fault detection method on SMGF and PMGF by incorporating an additional line in the original circuit to construct the augmented circuit to enable the circuit testability. The paper [11] in 2017 introduces a novel genetic algorithm-driven heuristic method for generating test sets aimed at fault detection within reversible circuits, eliminating the necessity for exhaustive searches. It presents two distinct approaches: one leveraging random search and the other employing directed search. These methodologies have been thoroughly validated using benchmark circuits, encompassing various fault types such as missing-gate faults (both complete and partial), bridging and stuck-at faults. In 2019, the authors in [12] proposed an efficient testing scheme that identifies the absence of gates from nearest neighbor circuits. The circuit is tested in this scheme by employing a DFT model. In a 2021 study referenced as [13], authors introduced a method for pinpointing faults and detecting them in MCT-based reversible circuits, encompassing all possible MGFs.

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.

Their approach involves analyzing both the primary circuit and its complement to identify faults. A comparator plays a pivotal role in fault detection by comparing the input of the reversible circuit with the output of its complement. In 2022, the authors in [14] introduced two DFT techniques equipped with a universal test set (UTS) for detecting MGFs in reversible circuits. These techniques involve transforming an $n \times n$ reversible circuit, implemented with k-CNOT gates, into a testable design by adding two extra input lines and additional gates.

From the above literature, it is noticeable that there needs to be more comprehensive methods to evaluate all MGFs using a single technique capable of generating a single test pattern. Existing approaches often entail intricate fault detection procedures involving circuit reconstruction or modification, consequently elevating circuit costs using test vectors. Therefore, we are inspired to introduce a simple yet impactful technique for detecting SMGFs, MMGFs, RGFs, and PMGFs without altering the original reversible circuit and by a unit test pattern.

III. Proposed Unity Method

This section introduces a method for constructing a comprehensive test pattern to identify SMGF, MMGF, RGF and PMGF in a k-CNOT reversible circuit. Initially, the process involves generating a complete test set specifically tailored to identify all MGFs. By generating a unity pattern of the logic value 1's depending upon the count of lines of the reversible circuit and converting the unity pattern to an augmented pattern by summing the unity pattern by the binary sequence $\langle 011 \rangle$, then reconstruct the augmented pattern by ignoring the least significant bit (LSB) and applying it to the last gate of the circuit which is further used in backtracking process till the first gate of the circuit. A test pattern is generated at the initial level of the circuit, which effectively detects all MGFs. The process flow of the unity method is depicted in Fig. 2. The suggested approach can broadly be described as follows:

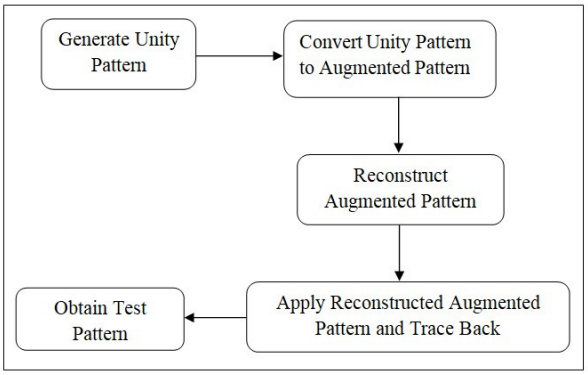

Fig. 2. Process flow of unity method

A. Generate unity pattern

At the very outset of the unity method process, generating a unity pattern is performed in the considered reversible circuit. The unity pattern is produced depending on the existence of the line count in the circuit. If the reversible circuit contains a line count of n, then the unity pattern will contain n number of binary 1's. Let us consider, a reversible circuit having input lines n=3, then, a unity pattern is formulated with the binary sequence $\langle 111 \rangle$. Similarly, the reversible circuits having the input lines n=4 and n=5, then a unity pattern is generated as $\langle 1111 \rangle$ and $\langle 11111 \rangle$, respectively.

B. Conversion process of unity pattern to an augmented pattern

The generated unity pattern is next converted to the augmented pattern. It is performed by adding binary sequence $\langle 11 \rangle$ to the unity pattern. Converting the unity pattern to an augmented pattern holds significant importance that is explained in detail below:

A circuit containing one input line has a unity pattern equal to $\langle 1 \rangle$, which is equivalent to 1 in decimal form. Now, whenever binary value 1 is assigned as an augmented value for the binary addition to the unity pattern for any circuit containing any n number of lines, then it always produces a binary value 0 in all the bit positions except the most significant bit (MSB). For instances, the augmented test patterns $\langle 1 \rangle$+$\langle 1 \rangle$=$\langle 10 \rangle$, $\langle 11 \rangle$+$\langle 01 \rangle$=$\langle 100 \rangle$, $\langle 111 \rangle$+$\langle 001 \rangle$=$\langle 1000 \rangle$, $\langle 1111 \rangle$+$\langle 0001 \rangle$=$\langle 10000 \rangle$ and so on. Similarly, a circuit containing two lines has a unity pattern that equals $\langle 11 \rangle$, which is equivalent to 3 in decimal form. Whenever the binary value $\langle 11 \rangle$ is added to the unity pattern, then it produces at least one non-zero bit excluding MSB. For example, $\langle 11 \rangle$+$\langle 11 \rangle$=$\langle 110 \rangle$, $\langle 111 \rangle$+$\langle 011 \rangle$=$\langle 1010 \rangle$, $\langle 1111 \rangle$+$\langle 0011 \rangle$=$\langle 10010 \rangle$ and so on. The main purpose for adding the binary sequence $\langle 11 \rangle$ to the unity pattern for constructing the augmented pattern is that at least one-zero bit must be present in the augmented pattern, such that the augmented pattern satisfies the maximum number of fault detection criteria for all types of faults in MGFs. Thus, the augmented pattern can be achieved by adding $\langle 11 \rangle$ binary sequence to the unity pattern of the considered circuit.

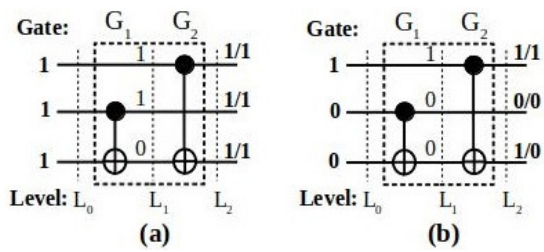

Fig. 3. Illustration of the first case of the fault-free/faulty circuits for MMGF at $\{G_1, G_2\}$: (a) apply unity test pattern (b) apply reconstructed augmented test pattern

It is observed that in certain cases the use of unity test pattern fails to identify all MGFs in reversible circuits. These cases are explained below.

In certain cases, the conversion process from a unity pattern to an augmented pattern is performed on account of the undetectable characteristics of some MGFs, such as MMGF and PMGF. In the first case, it is noticed that when the occurrence of an even number of gates have their target point in the same line, as depicted in Fig. 3 (a), then the test pattern is derived from the unity pattern fails to detect the MMGF, as shown in Fig. 3 (a). Here, the test pattern $\langle 111 \rangle$ is generated at the starting level L_0 by applying the

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.

backtracking process at the primary level L_2 of the circuit using the unity pattern $\langle 111 \rangle$. It is observed that when the test pattern $\langle 111 \rangle$ is used on the fault-free/faulty circuits, the fault-free/faulty primary outputs $\langle 111 \rangle$ /$\langle 111 \rangle$ are equivalent, as shown in Fig. 3 (a). Thus, the input test pattern $\langle 111 \rangle$ of the circuit that is derived from the unity pattern by applying the backtracking process fails to identify the fault-free and faulty circuits. In Fig. 3 (a), we consider the MMGF occurs at gates $\{G_1, G_2\}$, which is denoted as the dashed box. However, when it gets converted to an augmented pattern and the reconstructed augmented pattern (it is noted that the reconstructed augmented test pattern is explained in the next point) is used in the same circuit, then MMGF becomes detectable, as shown in Fig. 3 (b). Here, the initial input test pattern $\langle 100 \rangle$ at level L_0 is derived from the reconstructed augmented test pattern $\langle 101 \rangle$. The test pattern $\langle 100 \rangle$ at level L_0 is applied to the fault-free/faulty circuits, causes different fault-free/faulty primary outputs $\langle 101 \rangle$/$\langle 100 \rangle$ are generated for the fault-free/faulty circuits, as depicted in Fig. 3 (b). Therefore, the input test pattern $\langle 100 \rangle$ that is derived from the reconstructed augmented test pattern can be detected the faulty circuit.

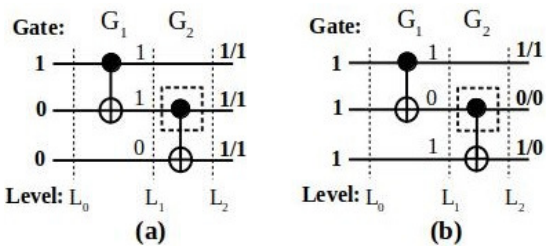

Fig. 4. Illustration of the second case of the fault-free/faulty circuits for PMGF at the control point of gate G_2: (a) apply unity test pattern (b) apply reconstructed augmented test pattern

The second case is found for the circuits containing gates of one control point and one target point in which the input line matches the previous gate's target point and the next gate's control point. This type of circuit fails to detect PMGF by the unity pattern, as shown in Fig. 4 (a). Here, the input test pattern $\langle 100 \rangle$ at level L_0 is inherited from the unity test pattern $\langle 111 \rangle$ using backtracking process is applied to the fault-free/faulty circuits. It is observed that the fault-free/faulty primary outputs $\langle 111 \rangle$ /$\langle 111 \rangle$, which are equivalent for the fault-free/faulty circuits. Thus, the input test pattern $\langle 100 \rangle$ of the circuit is unable to detect the PMGF fault at the control point (denoted as dashed box) of gate G_2, as shown in Fig. 4 (a). However, the input test pattern is derived from the reconstructed augmented test pattern using the backtracking process, which makes the PMGF detectable. In Fig. 4 (b), the input test pattern $\langle 111 \rangle$ that is derived from the reconstructed augmented test pattern $\langle 101 \rangle$ using backtracking process, which is generated the fault-free/faulty primary outputs $\langle 101 \rangle$ /$\langle 100 \rangle$ for the fault-free/faulty circuits.

C. Reconstruct augmented pattern and apply backtracking process

The augmented pattern is further reconstructed by ignoring the LSB of the augmented pattern that is obtained from the unity pattern to continue the stable n count in the considered circuit. For example, for a circuit with 4-input lines, the augmented test pattern becomes $\langle 10010 \rangle$. Thus, the reconstructed augmented pattern becomes $\langle 1001 \rangle$ by ignoring the LSB. The produced reconstructed augmented test pattern is applied to the final gate of a considered circuit and traced back to the initial level of the circuit. The generated test pattern at the initial level of the circuit is capable of identifying at least one fault of all MGFs containing SMGF, MMGF, RGF, and PMGF.

The following definitions are provided to explain the unity method with the help of Algorithm 1.

Definition 1. *A test pattern U_{pn} is the unity pattern that holds the binary inputs $\langle b_1 b_2 \ \ldots b_n \rangle$ in accordance with the count of input lines n contained in the circuit, where $1 \leq i \leq n$ and $b_i \in \{1\}$.*

Definition 2. *A test pattern A_{pn}=$\{\langle b_1 b_2 \ \ldots b_n b_{n+1} \rangle$ is the augmented test pattern that is generated by adding binary sequence $\langle 11 \rangle$ with U_{pn}, where $b_1 \in 1$ and $b_{n+1} \in 0$ are the MSB and LSB, respectively.*

Definition 3. *A test pattern RA_{pn}=$\{\langle b_1 b_2 \ \ldots b_n b_{n+1} \rangle \setminus \langle b_{n+1} \rangle | \langle b_1 b_2 \ \ldots b_n b_{n+1} \rangle \Rightarrow A_{pn}$, where $1 \leq i \leq n$ and $b_i \in \{0, 1\}\}$ is the reconstructed augmented test pattern that is derived from the generated augmented pattern A_{pn} by ignoring the least significant bit (LSB).*

Definition 4. *The test pattern TP=$\langle b_1 b_2 \ \ldots b_n \rangle$ is the initial input pattern of the circuit that is acquired via the reconstructed augmented test pattern RA_{pn} with the help of the backtracking process, which is finally used to identify the MGFs in reversible circuits.*

Algorithm 1: TEST PATTERN GENERATION FOR MGFS USING UNITY METHOD

Input: Reversible circuit with input line count n and gate count N.
Output: Generation of TP for detecting MGFs.

```
Generate U_pn by assigning logic value 1 to each n
  number of input lines
A_pn ← U_pn + ⟨11⟩
RA_pn ← (A_pn by discarding the LSB)
for j ← N to 1 do
    Apply RA_pn to G_N, N denotes the last gate
    Backtracking process is applied from G_N to G_1
      using RA_pn
Extract input test pattern TP at the initial level of G_1
```

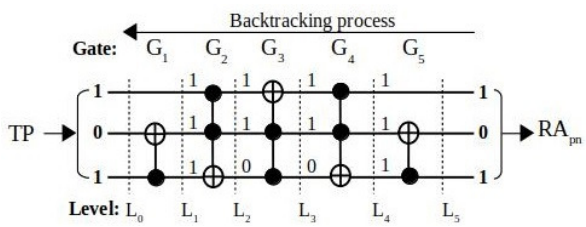

Fig. 5. Generation of TP by applying RA_{pn} using backtracking process in $miller$-11 circuit

Example 1. *Let us consider the benchmark circuit $miller$-11 to explain the detailed process flow of each step in*

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1. The $miller$-11 circuit comprises of five gates (N=5), which are denoted as G_1, G_2, G_3, G_4 and G_5 containing input line count n=3, as shown in Fig. 5. Each gate is categorized by the six levels L_0, L_1, L_2, L_3, L_4 and L_5. ***Step 1*** *shows the generation of the unity pattern based on the circuit's input line count n. As the considered circuit contains input lines n=3. Therefore, the unity test pattern U_{pn}=$\langle 111 \rangle$.* ***Step 2*** *defines the conversion of unity test pattern U_{pn}=$\langle 111 \rangle$ to an augmented test pattern A_{pn}. The augmented test pattern A_{pn} is generated by adding the binary sequence $\langle 011 \rangle$ to the U_{pn}. Thus, the generated A_{pn}=$\langle 111 \rangle$+ $\langle 011 \rangle$=$\langle 1010 \rangle$.* ***Step 3*** *reconstructs the augmented test pattern A_{pn} by discarding the LSB of A_{pn}. The newly generated reconstructed augmented test pattern is RA_{pn}=$\langle 101 \rangle$. In* ***Step 4****, consider all gates of the miller-11 circuit starting from the gate G_5 to G_1. In* ***Step 5****, the reconstructed augmented test pattern RA_{pn}=$\langle 101 \rangle$ is applied to the final gate G_5.* ***Step 6*** *applies to perform the backtracking process for each gate operation from the gate G_5 to G_1 by applying RA_{pn}=$\langle 101 \rangle$. In* ***Step 7****, test pattern TP=$\langle 101 \rangle$ is extracted at the initial level L_0, as depicted in Fig. 5, for the fault free $miller$-11 circuit. Here, the generated test pattern TP=$\langle 101 \rangle$ identifies various faults under the MGF model in the $miller$-11 circuit.*

Lemma 1. *The proposed unity method produces only a single distinct test pattern TP at the initial level in a reversible circuit.*

Proof. According to the proposed unity method, the reconstructed augmented test pattern RA_{pn} is applied to the Nth gate G_N. The backtracking process allows to perform each gate's operation from the final primary gate G_N to the initial gate G_1 in a given circuit. During the backtracking process, the controllability property [15] guarantees that a distinct test pattern TP is produced at the initial level of the circuit by applying a test pattern to any level of the circuit. Hence, the generated RA_{pn} is applied to the circuit using the backtracking process that is always feasible to produce a distinct test pattern TP at the initial level of the circuit in the proposed unity method. □

IV. Experimental Results

The empirical findings for identification of defects in reversible MCT-based circuits under the MGF model are presented in Table I. The unity method has been executed using Python 3.4 and executed on an Ubuntu v16.04 (64-bit) system running on a core-i3 device with 4 gigabytes of RAM. A total of 45 numbers of reversible benchmark circuits have been considered for examined that are available in [16], and these are placed in column 1 of Table I. The columns 2, 3 and 4 represent the quantity of input lines, the gate count, and the total number of existing MGFs, including SMGF, MMGF, RGF, and PMGF. Column five of Table I shows the number of detectable faults by the proposed unity method. Column 6 denotes the unit test pattern generation for detectable faults and the column 7 represents the total time consumption (in seconds) by CPU for running the algorithm for respective circuits. The circuit hwb5_13 is the highest number of input lines (28 nos), and the circuit $hwb7tc$ is the highest gate count (291 nos).

TABLE I
Experimental Results for identification of all MGFs by Unity method with CPU Time

Benchmark Circuit	n	N	Total no. of MGFs (SMGF+MMGF +RGF+PMGF)	Total no. of detectable MGFs [Proposed]	Test pattern (TP)	CPU Time (Sec)
cm82a_208	8	22	325	298	1	0.04
ex3_229	6	7	41	32	1	0.17
fredkin-6	3	3	15	10	1	0.00
graycode6_47	6	5	30	19	1	0.02
ham3tc	3	5	26	23	1	0.15
hwb4tc	4	17	107	86	1	0.01
hwb5_55	5	24	344	313	1	0.23
hwb7tc	7	291	43177	28932	1	9.35
hwb5_13	28	88	4135	3217	1	2.87
hwb5tc	5	56	1957	1483	1	1.01
hwb6tc	6	126	855	5328	1	2.01
majority_239	6	8	65	47	1	0.07
miller_11	3	5	28	17	1	0.02
mini-alu_84	4	20	257	193	1	0.46
mini-alu_167	4	6	48	36	1	0.15
mod10_171	4	10	101	71	1	0.21
mod5adders	6	21	185	143	1	0.14
mod5d1_63	5	7	56	29	1	0.08
mod5d2	5	9	74	65	1	0.13
mod5d1	5	8	60	47	1	0.23
mod8-10_177	5	14	159	119	1	0.03
nth_prime	3	4	19	9	1	0.01
peres	3	2	8	6	1	0.00
rd32-v0_66	4	4	18	15	1	0.21
rd53_137	7	16	213	194	1	0.09
rd53rcmg	7	30	570	456	1	1.96
rd53d2	8	12	113	96	1	0.82
rd53d1	7	12	145	105	1	2.72
rd84d1	15	28	598	345	1	5.13
rd73d2	10	20	292	213	1	1.92
sym6_145	7	36	952	731	1	0.18
xor5_254	6	7	69	54	1	0.05
xor5d1	5	4	19	12	1	0.34
ham7tc	7	24	304	267	1	0.22
ham15-109	15	109	6230	4319	1	3.41
3_17_3tc	3	6	34	25	1	0.01
3_17_6	7	17	190	154	1	0.32
4gt4-v0_78	5	13	122	96	1	0.25
9symd2	12	28	498	394	1	2.53
6symd2	10	20	315	286	1	2.02
2of5d1	6	18	279	213	1	1.51
5mo5tc	6	17	250	197	1	1.93
4_49tc1	4	16	166	134	1	1.74
2of5d2	7	12	121	112	1	1.62
rd32	4	4	19	17	1	0.01

TABLE II
Results of comparison in terms of no. of test pattern for identification of all MGFs by Unity method with [2]

Benchmark Circuit	n	N	Total no. of Test patterns (SMGF+PMGF) [2]	Total no. of Test patterns (SMGF+MMGF +RGF+PMGF) [Proposed]	Fault Coverage (%) [2]	Fault Coverage (%) [Proposed]
hwb7tc	7	291	8	1	100	67
hwb6tc	6	126	7	1	100	62.28
hwb5tc	5	56	6	1	100	75.78
rd53rcmg	7	30	8	1	100	80
rd84d1	15	28	16	1	100	57.69
9symd2	12	28	13	1	100	78.68
ham7tc	7	24	8	1	100	87.82
mod5adders	6	21	7	1	100	77.29
6symd2	10	20	11	1	100	90.79
rd73d2	10	20	11	1	100	72.94
2of5d1	6	18	7	1	100	76.34
5mod5tc	6	17	7	1	100	78.80
hwb4tc	4	17	5	1	100	80.37
4_49tc1	4	16	5	1	100	80.72
rd53d2	8	12	9	1	100	84.95
rd53d1	7	12	8	1	100	72.41
2of5d2	7	12	8	1	100	92.56
mod5d2	5	9	6	1	100	87.83
mod5d1	5	8	6	1	100	78.33
3_17_13tc	3	6	4	1	100	73.52
ham3tc	3	5	4	1	100	88.46
xor5d1	5	4	6	1	100	63.15
rd32	4	4	5	1	100	89.47

Table II shows the comparative results for identification of all MGFs by unity method with existing work as explained in [2]. Columns 4 and 5 of Table II provide the total number of test patterns generated for detecting the faults by both existing work in [2] and the proposed method, respectively. The fault identification rate for both work in [2]

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.

and proposed method have been illustrated in columns 6 and 7, respectively. In accordance with comparative study, it is found that the proposed method requires only a single test pattern to detect the various types of faults in the MGF model, as compared to the existing method [2]. However, the fault coverage range of the proposed method is less than the existing method, but the proposed method has occupied all types of MGFs. Whereas the work in [2] does not considered the MMGFs.

V. Conclusion

In this paper, a fault recognition approach has been established that is aimed at detecting MGFs in reversible circuits. This method provides the unity method, which offers a streamlined approach to identifying various types of MGFs, including SMGF, MMGF, RGF, and PMGF. The core of our approach involves generating a U_{pn} by introducing the logic value 1 to each input line, which is enhanced by A_{pn} and RA_n. Next, RA_{pn} is fed into the last gate of the circuit. By subsequently executing a backtracking process from the rearmost gate to the foremost gate of the circuit, we derive a test pattern crucial for pinpointing any underlying faults. This methodology not only enhances fault detection accuracy but also significantly reduces computational overhead by finding only one test pattern.

References

[1] I. Polian, T. Fiehn, B. Becker, and J. P. Hayes,"A family of logical fault models for reversible circuits," IEEE Asian Test Symposium, pp 422–427, 2005.

[2] H. Rahman, D. K. Kole, D. K. Das and B. B. Bhattacharya, "Fault diagnosis in reversible circuits under missing-gate fault model," Computers & Electrical Engineering, vol. 37, no. 4, pp. 475–485, 2011.

[3] B. Mondal, P. Das, P. Sarkar and S. Chakraborty, "A comprehensive fault diagnosis technique for reversible logic circuits," Computers & Electrical Engineering, vol. 40, no. 7, pp. 2259–2272, 2014.

[4] J. P. Hayes, I. Polian, and B. Becker, "Testing for missing-gate faults in reversible circuits," IEEE Asian Test Symposium, pp. 100–105, 2004.

[5] H. M. Gaur, A. K. Singh and U. Ghanekar, "Offline testing of reversible logic circuits: an analysis," Integration, 62, 2018, pp. 50–67, 2018.

[6] J. Zhong, and J. C. Muzio, "Analyzing fault models for reversible logic circuits," in 2006 IEEE international conference on evolutionary computation, pp. 2422–2427, IEEE, 2006.

[7] X. Fang-ying, C. Han-wu, L. Wen-jie and L. Zhi-giang, "Fault detection for single and multiple missing-gate faults in reversible circuits," in 2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence), pp. 131–135, 2008.

[8] M. A. Nielsen and I. L. Chuang, "Quantum computation and quantum information," Phys. Today, vol. 54, no. 2, p. 60, 2001.

[9] B. Mondal, D. K. Kole, D. K. Das and H. Rahaman, "Generator for test set construction of SMGF in reversible circuit by Boolean difference method," in 2014 IEEE 23rd Asian test symposium, IEEE, pp. 68–73, 2014.

[10] B. Mondal, C. Bandyopadhyay and H. Rahaman, "A testing scheme for mixed-control based reversible circuits," in 2016 Sixth International Symposium on Embedded Computing and System Design (ISED), pp. 96–100, 2016.

[11] A. Nagamani, S. Anuktha, N. Nanditha, and V. K. Agrawal, "A genetic algorithmbased heuristic method for test set generation in reversible circuits," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 37, no. 2, pp. 324–336, 2017.

[12] A. Bhattacharjee, C.Bandyopadhyay, B. Mondal, and H. Rahaman, "A novel fault-detection scheme for nearest-neighbor-based reversible circuits," in Soft Computing: Theories and Applications: Proceedings of SoCTA 2019, pp. 489–500, Springer Singapore, 2019.

[13] D. Kheirandish, M. Haghparast, M. Reshadi and M. Hosseinzadeh, "Efficient techniques for fault detection and location of multiple controlled Toffoli-based reversible circuit," Quantum Information Processing, vol. 20, no. 11, p. 370, 2021.

[14] J. Mondal, D. K. Kole, H. Rahaman, D. K. Das, B. B. Bhattacharya, "DFT with Universal Test Set for All Missing Gate Faults in Reversible Circuits," Journal of Circuits, Systems and Computers, vol. 31, no. 10, p. 2250128, 2022.

[15] K. N. Patel, J. P. Hayes, and I. L. Markov, "Fault testing for reversible circuits," Computer-Aided Design of Integrated Circuits and Systems,IEEE Transactions, vol. 23, no. 8, 2004, pp. 1220–1230.

[16] D. Maslov, "Reversible logic synthesis benchmarks page, 2011," URL http://webhome.cs.uvic.ca/ dmaslov, 2020.

[17] M. Handique, J. K. Deka, and S. Biswas, "An Efficient Test Set Construction Scheme for Multiple Missing-Gate Faults in Reversible Circuits," Journal of Electronics Testing, vol. 36, 2020, pp. 105–122.

Authorized licensed use limited to: SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECH. Downloaded on August 05,2025 at 10:02:28 UTC from IEEE Xplore. Restrictions apply.